LZMA Compression Explained in Detail

From information theory and the range coder
to every bit of the packet format

A self-contained tutorial with worked examples,
verified against a reference implementation

About the examples. Every numeric example in this document (range-coder arithmetic, probability tables, the token trace for the 15-byte sample string, the benchmark table) was computed by running code, and the C decoder in Appendix A was tested against files produced by the real `xz` tool (liblzma), including text, executable code, random data, long runs, an empty file, and several `lc`/`lp`/`pb` settings, under AddressSanitizer and UBSan.

Contents

How to read this document

LZMA is the algorithm behind the `.7z` and `.xz` formats. It routinely produces files that are 20–40% smaller than those produced by Deflate (`gzip`, `zip`, `zlib`) and often smaller than `bzip2`. The reason is not one clever trick but a sequence of well-chosen engineering decisions, each of which saves a little, and which compound. This document walks through all of them.

The document is organised in three layers:

Foundations (Chapters 1–4)
Entropy, dictionary (LZ77) compression, and the *range coder*, which is the engine that makes LZMA special. If you already know these, skim Chapter 4 for the exact integer arithmetic.

The LZMA format (Chapters 5–10)
The packet types, the state machine, literal coding, length and distance coding, repeated matches, a complete end-to-end trace of a real compressed stream, and the container formats.

The encoder and the big picture (Chapters 11–12)
How an encoder chooses packets (match finders, optimal parsing), and a quantitative account of why LZMA beats the alternatives and where it loses.

Chapter 13 contains exercises (with solutions), and Appendix A contains a complete, tested LZMA decoder in about 200 lines of C. Reading that code after Chapters 4–8 is the best way to check that you really understood everything: the decoder is the specification.

Conventions. Bits are written `like this`. Hexadecimal numbers carry a `0x` prefix. "Byte" means 8 bits. $\log_2$ is the base-2 logarithm. Positions in the data are counted from 0. Probabilities inside LZMA are integers in $[0, 2048)$; when we say "probability p" for such a number we mean $p/2048$.

Chapter 1

Introduction

1.1 What is LZMA?

LZMA stands for *Lempel–Ziv–Markov chain algorithm*. It was designed by Igor Pavlov and first appeared in the 7-Zip archiver around 2001. Its reference implementation, the LZMA SDK, is in the public domain. Today it is found in the `.7z` archive format, in the `.xz` and `.lzma` file formats, and in many software distribution and kernel-image formats, where one compresses once and decompresses many times.

The name describes two ingredients:

- **Lempel–Ziv**: a *dictionary* method. The compressor replaces repeated byte strings by references *(distance, length)* back into data it has already emitted.
- **Markov chain**: the compressed symbols are coded using *context-dependent probabilities*, where the context includes a small state machine that remembers the last few kinds of packets. This is what lets the encoder predict whether the next thing is likely to be a literal byte or a match.

The one-paragraph summary

LZMA first turns the input into a stream of *packets*: **literals** (a raw byte), **matches** (copy L bytes from distance D back), and **repeated matches** (reuse one of the four most recent distances). It then encodes every single decision ("is this a match?", "which length?", each bit of each byte, ...) as a *binary decision with an adaptive probability*, using a **range coder** that spends fractions of a bit per decision when the decision is predictable. The context used to look up each probability is carefully chosen so predictions are sharp.

1.2 The two-layer architecture

Figure **??** shows the pipeline. Everything in LZMA belongs to one of two layers, and keeping them mentally separate is the key to understanding it.

Layer 1 (parsing) is a search problem: find a good way to describe the data as literals and matches. The format does not dictate *how* the encoder does this. Any valid packet sequence decodes correctly, so encoders are free to trade speed for quality. That freedom is why `xz -0` and `xz -9` produce streams that every decoder can read.

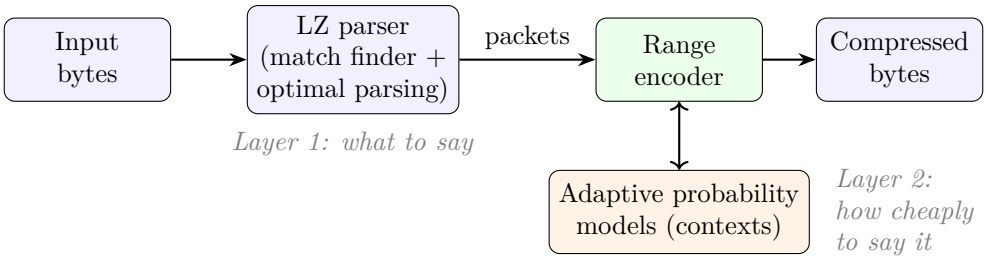


Figure 1.1: The LZMA pipeline. Layer 1 decides *what* to encode (a sequence of literals and matches). Layer 2 decides *how many bits* each decision costs, by modelling probabilities adaptively.

Layer 2 (entropy coding) is a modelling problem, and it is rigidly specified by the format, because the decoder must reproduce exactly the same probabilities as the encoder.

1.3 What makes it "beat" other algorithms?

A preview of the answer (made quantitative in Chapter 12):

1. A **huge dictionary**: Deflate looks back at most 32 KiB; LZMA can look back megabytes to gigabytes.
2. A **range coder with adaptive binary probabilities**: no whole-bit rounding (Huffman needs at least 1 bit per symbol) and no side information (Huffman trees must be transmitted; LZMA's models are learned on the fly by both sides).
3. **Rich contexts**: a literal is coded knowing the previous byte; a flag is coded knowing the recent history of packet types and the position modulo a small power of two.
4. **Matched literals**: right after a match, the byte that *would have* continued the match is used as extra context for the next literal.
5. **Repeated-distance packets**: the last four distances are cheap to reuse, which fits structured data (tables, code, records).
6. **Optimal parsing**: the encoder runs a dynamic-programming shortest-path search over the cost of alternative packet sequences, rather than greedily taking the longest match.

The price is speed: compression is slow and decompression, while fast enough for most uses, is inherently sequential and slower than Deflate or Zstandard.

Chapter 2

Information Theory in One Chapter

You do not need much theory to understand LZMA, but a few ideas appear everywhere in the rest of this document.

2.1 Information and entropy

If an event has probability p, observing it conveys $-\log_2 p$ bits of *information* (surprise). Rare events are surprising and therefore expensive to communicate; near-certain events cost almost nothing.

p	cost $-\log_2 p$ (bits)
0.5	1.000
0.9	0.152
0.985	0.022
0.1	3.322
0.01	6.644

The *entropy* of a source is the expected cost per symbol, $H = -\sum_i p_i \log_2 p_i$. Shannon's source coding theorem says no lossless compressor can beat H bits per symbol on average *for a source with those probabilities*, and that one can get arbitrarily close.

Worked example: a biased coin

A coin shows heads with probability 0.1. Then

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.9 \cdot 0.152 + 0.1 \cdot 3.322 = 0.469 \text{ bits per flip.}$$

A naive encoding uses 1 bit per flip; an ideal coder needs only 0.469 bits per flip, a saving of 53%.

2.2 Compression is prediction

An ideal coder spends $-\log_2 p$ bits for a symbol the model predicted with probability p. So the quality of compression is exactly the quality of the *predictions*. A compressor therefore has two independent jobs:

Modelling assign a probability to every possible next event (using whatever the encoder and decoder both know).

Coding turn probabilities into bits with as little waste as possible.

Huffman coding is a coding method with a significant limitation: each symbol gets a whole number of bits. If a binary decision has $P(\text{yes}) = 0.99$, the entropy is $H = 0.081$ bits, but Huffman still spends a full bit per decision, a *twelve-fold* overhead. LZMA codes a large number of very skewed binary decisions ("is the next packet a literal?" is true or false with high confidence in many contexts), so a coder that can spend 0.02 bits on a predictable decision is decisive. That coder is the **range coder** of Chapter 4.

Key idea

Huffman coding is wasteful for highly predictable binary decisions, because it cannot spend less than one bit per symbol. Arithmetic/range coding has no such floor, and LZMA is designed to exploit this.

Chapter 3

Dictionary Compression (LZ77)

3.1 The basic idea

Real data repeats itself: words in text, identifiers in source code, instruction sequences in executables. LZ77 (Ziv and Lempel, 1977) exploits this by keeping a *window* of recently processed data (the *dictionary*) and replacing a repeated string by a pair (D, L) meaning "go back D bytes and copy L bytes".

Worked example: LZ77 on `abracadabra`

Positions: `a b r a c a d a b r a` are positions $0, \ldots, 10$. Scanning left to right, the encoder emits literals for `a b r a c a d`. (At position 3, the byte `a` matches position 0 but a length-1 match is not worth a reference.) At position 7 the string `abra` also occurs at position 0, i.e. $D = 7$ bytes back, with $L = 4$:

$$\underbrace{\texttt{a b r a c a d}}_{\text{7 literals}} \quad \underbrace{(D{=}7, L{=}4)}_{\text{match}}$$

Eleven bytes have become 7 literals plus one match. The decoder rebuilds the data by copying 4 bytes starting 7 bytes behind its write position.

3.2 Overlapping matches and run-length behaviour

A match is allowed to have $L > D$. The decoder copies *byte by byte*, so bytes that were just written can be copied again:

```
function copy_match(out, D, L):
    for i in 0 .. L-1:
        out.append( out[ len(out) - D ] )     // reads bytes written a moment ago
```

For example, `aaaaaaaa` (8 bytes) is the literal `a` followed by the match $(D{=}1, L{=}7)$. A distance-1 match therefore behaves as run-length encoding, and a distance-k match repeats a k-byte pattern. No separate RLE mechanism is needed.

3.3 Parameters of the LZMA dictionary

	Deflate	LZMA
Window (max distance)	32 KiB	up to 4 GiB in the format (typically 1–64 MiB in practice)
Minimum match length	3	2
Maximum match length	258	273

The size of the window matters a great deal for large files with long-range redundancy (for example, similar source files or sections of a binary far apart). A match finder for such a window needs good data structures (Chapter 11).

3.4 Parsing: which matches should be used?

At each position many candidate matches exist (different distances and lengths). The *parsing strategy* chooses among them:

Greedy always take the longest match available. Fast, but often suboptimal.

Lazy look one position ahead and delay if a longer match starts at the next byte. (Used by gzip.)

Optimal consider the *cost in bits* of alternatives over a window of positions and choose the cheapest path. This is what LZMA's best compression modes do (Chapter 11).

3.5 Why LZ77 alone is not enough

Plain LZ77 outputs a stream of literals and (D, L) pairs. Written naively (say, 1 flag bit, 8 bits per literal, and 16+8 bits per match) it compresses only modestly. All the sophistication of modern LZ-based compressors lies in how the packets are *coded*. Deflate applies Huffman coding to the literal/length symbols and distance symbols, with trees transmitted per block. LZMA instead uses a context-modelled adaptive range coder and a richer set of packet types. The next chapter builds that coder.

Chapter 4

The Range Coder

This is the most important chapter. Once you understand it, the LZMA format is just a long list of *which probability to use for which decision*.

4.1 Arithmetic coding in one picture

An arithmetic coder represents an entire message as a single number inside a nested sequence of shrinking intervals. Start with the interval $[0, 1)$. For each binary decision, split the current interval in proportion to the probabilities of the two outcomes and keep the part matching the actual outcome. The final interval is narrow; any number inside it identifies the message.

Worked example: decimal toy coder

Let $P(\text{bit} = 0) = 0.8$. Encode the bits $0, 0, 1$.

step	bit	interval before	interval after
1	0	[0, 1)	[0, 0.8)
2	0	[0, 0.8)	[0, 0.64)
3	1	[0, 0.64)	[0.512, 0.64)

The final width is $0.8 \cdot 0.8 \cdot 0.2 = 0.128$, so the message costs $-\log_2 0.128 = 2.97$ bits. The binary fraction $0.1001_2 = 0.5625$ lies inside $[0.512, 0.64)$ and needs only 4 bits to write down, so a number of about $\lceil \log_2(1/0.128) \rceil + 1 = 4$ bits identifies the message. The cost is the information content, up to a small constant, with no per-symbol rounding.

A **range coder** is an arithmetic coder implemented with integer arithmetic, working on bytes: it keeps a *low* end and a *range* (width) in 32-bit/64-bit integers and emits bytes as soon as they are certain. LZMA uses a *binary* range coder: every decision is a single yes/no with an adaptive probability.

4.2 Adaptive probabilities

LZMA stores each probability as an 11-bit integer `prob` in $[0, 2048)$: the probability that the next bit is `0` is `prob`/2048. Each probability starts at 1024 (that is, 1/2), and after every coded

bit it moves towards the observed outcome:

$$\begin{aligned} \text{bit} = 0: &\quad \texttt{prob} \leftarrow \texttt{prob} + (2048 - \texttt{prob}) \gg 5, \\ \text{bit} = 1: &\quad \texttt{prob} \leftarrow \texttt{prob} - (\texttt{prob} \gg 5). \end{aligned}$$

This is an exponentially weighted moving average with factor 31/32: the model effectively remembers roughly the last 32 occurrences in that context.

Worked example: how fast does a probability learn?

Starting from $\texttt{prob} = 1024$ and seeing only zeros, $\texttt{prob}_n \approx 2048 - 1024\,(31/32)^n$. The table below was computed by actually iterating the integer update rule.

zeros seen	prob	$P(0)$	cost of next 0 (bits)
0	1024	0.500	1.000
1	1056	0.516	0.956
2	1087	0.531	0.914
5	1174	0.573	0.803
10	1300	0.635	0.656
20	1499	0.732	0.450
50	1828	0.893	0.164
100	1990	0.972	0.041
≥ 200	2017	0.985	0.022

A built-in floor

Because of the integer shifts, prob can never exceed $2048-31 = 2017$ (once $2048-\texttt{prob} < 32$ the increment $\lfloor(2048-\texttt{prob})/32\rfloor$ is 0), nor drop below 31. So even a perfectly predictable bit costs at least $-\log_2(2017/2048) \approx 0.022$ bits. This is not a bug: a probability that could reach 1 would make the first surprise catastrophically expensive.

4.3 The encoder

State: low (a 64-bit integer, only 33 bits meaningful), range (32-bit), plus a cache byte and a counter cacheSize used for carry propagation. Initially low = 0, range = 0xFFFFFFFF, cache = 0, cacheSize = 1.

Listing 4.1: Encoding one bit with an adaptive probability.

```
function encode_bit(prob, bit):
    bound = (range >> 11) * prob          // size of the "bit = 0" sub-interval
    if bit == 0:
        range = bound
        prob  = prob + ((2048 - prob) >> 5)
    else:
        low   = low + bound               // skip past the "0" part
        range = range - bound
        prob  = prob - (prob >> 5)
    while range < 2^24:                   // normalize: keep range large
        range = range << 8
        shift_low()
```

The probability occupies the *lower* part of the interval (the `0` outcome) and the `1` outcome the remainder. Note that `range` $\gg 11$ first discards 11 low bits so the product with an 11-bit `prob` cannot overflow 32 bits.

Listing 4.2: Emitting bytes and propagating carries.

```
function shift_low():
    if low < 0xFF000000 or low >= 2^32:          // top byte is now certain (or a carry
    occurred)
        carry = low >> 32                         // 0 or 1
        temp  = cache
        do:
            output_byte( (temp + carry) & 0xFF )
            temp = 0xFF
        while --cacheSize != 0
        cache = (low >> 24) & 0xFF
    cacheSize += 1
    low = (low & 0x00FFFFFF) << 8
```

Why the cache? Adding `bound` to `low` can overflow into bit 32 (a *carry*) that must be added to a byte that may already have been "almost" emitted. The coder therefore holds back the most recent byte (`cache`) together with a run of `0xFF` bytes (`cacheSize`-1 of them), since only those can change if a carry arrives. This also explains a curious fact: **the first byte of every LZMA range-coded stream is `0x00`**, because the very first byte shifted out is the initial `cache`.

At the end the encoder calls `shift_low()` five times to flush everything.

4.4 The decoder

The decoder mirrors the encoder but keeps `code`, the difference between the received number and `low`. Initially it reads 5 bytes (the first must be `0x00`), setting `range = 0xFFFFFFFF` and `code` to the next 4 bytes read big-endian.

Listing 4.3: Decoding one bit.

```
function decode_bit(prob):
    bound = (range >> 11) * prob
    if code < bound:                        // falls in the "0" part
        range = bound
        prob  = prob + ((2048 - prob) >> 5)
        bit = 0
    else:
        range = range - bound
        code  = code - bound
        prob  = prob - (prob >> 5)
        bit = 1
    if range < 2^24:                        // normalize
        range = range << 8
        code  = (code << 8) | next_input_byte()
    return bit
```

The decoder only needs to *compare* `code` with `bound`. It performs exactly the same `prob` and `range` updates as the encoder, which is why the two stay synchronised without ever communicating probabilities.

4.5 Numeric trace

Let us encode the three bits $0, 0, 1$ starting from a fresh probability `prob` $= 1024$ and `range` $=$ `0xFFFFFFFF` $= 4\,294\,967\,295$. (The following values were produced by running the algorithm.)

bit	`prob` before	`bound`	`range` after	`low` after	`prob` after
0	1024	2 147 482 624	2 147 482 624	0	1056
0	1056	1 107 295 200	1 107 295 200	0	1087
1	1087	587 709 377	519 585 823	587 709 377	1054

Step 1: `range` $\gg 11 = 2\,097\,151$ and $2\,097\,151 \times 1024 = 2\,147\,482\,624$. Step 2 uses the new probability 1056: $(2\,147\,482\,624 \gg 11) \times 1056 = 1\,048\,575 \times 1056 = 1\,107\,295\,200$. Step 3: the bit is `1`, so `low` jumps up by `bound` and the new range is the remainder, $1\,107\,295\,200 - 587\,709\,377 = 519\,585\,823$. No normalisation was needed since the range is still above $2^{24} = 16\,777\,216$.

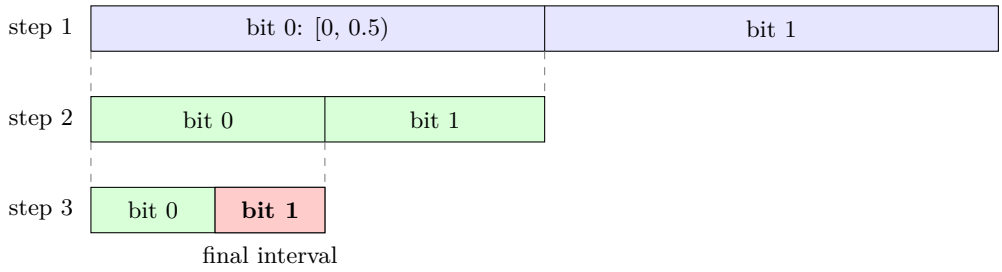


Figure 4.1: Interval narrowing for the bits $0, 0, 1$ (drawn to scale: the full bar is 2^{32} wide). At each step the current interval is split at `bound`; the part matching the coded bit becomes the next interval. The final interval is about 12.1% of the original, i.e. 3.05 bits of information, which is exactly the sum of the three modelled costs $1.000 + 0.956 + 1.092$ (the third bit is a "surprise" because the model had learned to expect a `0`).

4.6 Normalisation and precision

The coder maintains `range` $\geq 2^{24}$ after every step. Since `prob` $\in [31, 2017]$ and `range` $\gg 11 \geq 2^{13}$, both sub-intervals `bound` and `range` $-$ `bound` are non-empty. The use of `range` $\gg 11$ truncates a little, which costs a tiny amount of efficiency (relative error below 2^{-13} per decision); in return everything fits in 32-bit multiplication.

When the range falls below 2^{24} the coder shifts a byte out (encoder) or in (decoder). On average the stream therefore grows exactly as fast as the information content requires, in units of whole bytes but with *fractional bits* accumulated across decisions in `range`.

4.7 Coding multi-bit values

The binary coder codes one bit at a time. For larger values LZMA uses three constructions.

4.7.1 Bit trees

To code an m-bit value from most significant bit to least, use 2^m probabilities arranged as a binary tree. The index starts at `m = 1`; after coding a bit b, set `m` $\leftarrow$ `2m` $+ b$. The probability

used for each bit is `probs[m]` *before* the update. Because the index records all the earlier bits, each bit is predicted given the bits above it, so the tree learns an arbitrary distribution over the 2^m values.

Worked example: bit tree, 3 bits, value 5

$5 = \texttt{101}$. Start $\texttt{m} = 1$.

bit	probability used	new `m`
1	`probs[1]`	3
0	`probs[3]`	6
1	`probs[6]`	13

Finally the value is $\texttt{m} - 2^3 = 13 - 8 = 5$. Different values touch different probabilities, so common values become cheap.

4.7.2 Reverse bit trees

Identical, except that the bits are coded from the *least* significant bit to the most significant. Coding value $6 = \texttt{110}$: bits $0, 1, 1$ use `probs[1]`, `probs[2]`, `probs[5]` ($\texttt{m} : 1 \to 2 \to 5 \to 11$), and the value is rebuilt as $\sum b_i 2^i$. LZMA uses these for the low bits of distances, where the low bits are better predicted from each other than from the high bits.

4.7.3 Direct bits

Some bits are essentially random (the middle bits of large distances). These are coded with a fixed probability of $1/2$, with no model and no adaptation:

```
function encode_direct_bits(value, n):
    for i from n-1 down to 0:
        range = range >> 1
        if (value >> i) & 1: low = low + range
        normalize()
```

Each such bit costs exactly one bit.

Two meanings of "bit"

"Bit" can mean a bit of the data being compressed or a binary decision being coded. For the range coder there is no difference: every decision, whether a flag, a bit of a byte, or a bit of a length, is just "code this bit using this probability slot". Everything LZMA does is therefore about *choosing the probability slot*.

Chapter 5

LZMA Packets, State and Contexts

5.1 The packet types

An LZMA stream (after range decoding) is a sequence of packets. There are exactly seven kinds, distinguished by a short prefix of binary decisions, *each of which is coded with its own adaptive probability*:

decisions	packet	meaning
0	LIT	one literal byte follows
1 0	MATCH	new match: length, then distance (stored as $D-1$)
1 1 0 0	SHORTREP	one byte copied from distance `rep0`
1 1 0 1	LONGREP[0]	match of length ≥ 2 at distance `rep0`
1 1 1 0	LONGREP[1]	match at distance `rep1`
1 1 1 1 0	LONGREP[2]	match at distance `rep2`
1 1 1 1 1	LONGREP[3]	match at distance `rep3`

The decisions are made, in order, with these probability arrays: `isMatch`, `isRep`, `isRepG0`, `isRep0Long`, `isRepG1`, `isRepG2`. ("G" stands for "greater than": `isRepG0`=1 means "not rep0".)

Listing 5.1: Decision tree for choosing a packet type (decoder view).

```
if decode_bit(isMatch[state][posState]) == 0:           -> LITERAL
else if decode_bit(isRep[state]) == 0:                  -> MATCH
else if decode_bit(isRepG0[state]) == 0:
    if decode_bit(isRep0Long[state][posState]) == 0:    -> SHORTREP
    else:                                               -> LONGREP[0]
else if decode_bit(isRepG1[state]) == 0:                -> LONGREP[1]
else if decode_bit(isRepG2[state]) == 0:                -> LONGREP[2]
else:                                                   -> LONGREP[3]
```

Notice the Huffman-like shape: the most common packets (literals, new matches) are reached with the fewest decisions, but the whole tree is adaptive. If a file has many LONGREP[1] packets, those probabilities become sharp and they become cheap.

5.2 The state variable

The decisions above are conditioned on a **state** $\in \{0, \ldots, 11\}$, which summarises the last few packet types. It is a tiny Markov chain (hence the "M" in LZMA). Its purpose is to allow the model to learn statements like "after two literals, another literal is likely" or "right after a match, a literal is likely, and it is likely to differ from what the match would have predicted".

State transitions depend only on the packet just coded:

$$\text{after LIT:} \quad s \mapsto \begin{cases} 0 & s < 4 \\ s - 3 & 4 \le s < 10 \\ s - 6 & s \ge 10 \end{cases}$$

$$\text{after MATCH:} \quad s \mapsto \begin{cases} 7 & s < 7 \\ 10 & s \ge 7 \end{cases}$$

$$\text{after LONGREP:} \quad s \mapsto \begin{cases} 8 & s < 7 \\ 11 & s \ge 7 \end{cases}$$

$$\text{after SHORTREP:} \quad s \mapsto \begin{cases} 9 & s < 7 \\ 11 & s \ge 7 \end{cases}$$

s	recent history (oldest → newest)	LIT	MATCH	LONGREP	SHORTREP
0	LIT, LIT	0	7	8	9
1	MATCH, LIT, LIT	0	7	8	9
2	REP, LIT, LIT	0	7	8	9
3	SHORTREP, LIT, LIT	0	7	8	9
4	MATCH, LIT	1	7	8	9
5	REP, LIT	2	7	8	9
6	SHORTREP, LIT	3	7	8	9
7	LIT, MATCH	4	10	11	11
8	LIT, LONGREP	5	10	11	11
9	LIT, SHORTREP	6	10	11	11
10	non-LIT, MATCH	4	10	11	11
11	non-LIT, REP	5	10	11	11

Rule of thumb: states 0–6 mean "the last packet was a literal"; states 7–11 mean "the last packet was a match or rep". This split matters in Chapter 6 (matched literals) and in the updates above.

5.3 Position state: `pb`

Many files have structure at power-of-two alignments (32-bit words, 16-byte records, text lines padded to a width). LZMA conditions the flags and lengths on $\texttt{posState} = \text{pos} \,\&\, (2^{\texttt{pb}} - 1)$, where pos is the number of bytes decoded so far (the position of the *start* of the packet). The default is $\texttt{pb} = 2$ (four position classes).

5.4 The three tunable parameters

The format has three parameters, encoded in one properties byte:

$$\texttt{props} = (\texttt{pb} \times 5 + \texttt{lp}) \times 9 + \texttt{lc}.$$

param	default	meaning
`lc`	3	number of high bits of the *previous byte* used as literal context ($0 \le \texttt{lc} \le 8$)
`lp`	0	number of low bits of the *position* used as literal context ($0 \le \texttt{lp} \le 4$)
`pb`	2	number of low bits of the position used as `posState` ($0 \le \texttt{pb} \le 4$)

The default properties byte is $(2 \cdot 5 + 0) \cdot 9 + 3 = 93 = \texttt{0x5D}$, the value you will see as the first byte of nearly every `.lzma` file. Typical tuning: keep the defaults for general data; for 32-bit aligned binary data a common suggestion is $\texttt{lc} = 0$, $\texttt{lp} = 2$, $\texttt{pb} = 2$. These are only hints; the right values depend on the data and are best found by measuring.

Chapter 6

Literals

6.1 Plain literals

A literal is a byte coded with an 8-level **bit tree** (Chapter 4), with a *separate tree for every literal context*. The context is

$$\texttt{litState} = ((\text{pos} \,\&\, (2^{\texttt{lp}} - 1)) \ll \texttt{lc}) \;+\; (\text{prevByte} \gg (8 - \texttt{lc})).$$

With the defaults ($\texttt{lc} = 3$, $\texttt{lp} = 0$) this is simply the top 3 bits of the previous byte, giving 8 contexts. Each context has $\texttt{0x300} = 768$ probabilities (a 256-leaf tree needs 255 nodes; the rest is used by matched literals, below). In English text, the top bits of the previous byte distinguish, for example, lowercase letters from spaces and punctuation, which helps predict the next byte's class.

Why only 3 high bits?

Using the full previous byte would give 256 contexts of 768 probabilities each (≈196 608 probabilities): too many to learn from a file of ordinary size, because every context starts at 50/50 and must be trained. Three bits is a compromise between precision and learning speed. That is why `lc` is a tunable parameter.

6.2 Matched literals

Suppose the previous packet was a match (state ≥ 7) and the next byte is a literal. Intuitively: the match ended because the byte at the copy source differs from the actual byte. But usually they are *similar* (differing in just a few low bits, as with consecutive characters or numbers). Let `matchByte` be the byte at distance `rep0` + 1, i.e. what the match *would* have produced next.

LZMA uses `matchByte` as extra context while coding the literal bit by bit from the top:

Listing 6.1: Matched literal decoding (state ≥ 7).

```
sym = 1
while sym < 0x100:
    matchBit = (matchByte >> 7) & 1;  matchByte = matchByte << 1
    bit = decode_bit( probs[ ((1 + matchBit) << 8) + sym ] )
    sym = (sym << 1) | bit
    if matchBit != bit: break              // from here on, matchByte is useless
while sym < 0x100:                          // finish with the ordinary tree
    sym = (sym << 1) | decode_bit( probs[sym] )
byte = sym & 0xFF
```

The probability index has three disjoint regions: `sym` alone (plain tree, indices < 256), $256 +$ `sym` (the expected bit was `0`) and $512 +$ `sym` (expected `1`). So while the literal agrees with `matchByte`, each bit is predicted "given that the expected bit is m", which becomes a sharp prediction ("the bit equals the match bit"). After the first disagreement the coder falls back to an ordinary tree.

Worked example: matched literals in the sample string

In the string `abcdXabcdYabcdZ` (Chapter 9) the match `abcd` at position 5 is followed by the literal `Y`, while the byte after the source copy is `X`. The two bytes are `0x59` = `01011001` and `0x58` = `01011000`.

bit #	`sym` so far	expected (matchBit)	actual	`probs` index	mode
7 (MSB)	`1`	0	0	$256 + 1 = 257$	matched
6	`10`	1	1	$512 + 2 = 514$	matched
5	`101`	0	0	$256 + 5 = 261$	matched
4	`1010`	1	1	$512 + 10 = 522$	matched
3	`10101`	1	1	$512 + 21 = 533$	matched
2	`101011`	0	0	$256 + 43 = 299$	matched
1	`1010110`	0	0	$256 + 86 = 342$	matched
0 (LSB)	`10101100`	0	**1**	$256 + 172 = 428$	mismatch, done

Seven bits were "predicted" from the match byte. In a real file, after many matches those probability slots would hold strong beliefs like "when the expected bit is 1 the actual bit is almost always 1", and the cost of those seven bits becomes a few hundredths of a bit each.

Worked example: a mismatch earlier in the byte

The final literal `Z` (`0x5A` = `01011010`) follows a rep match whose source continuation is `Y` (`01011001`). The two agree for the first 6 bits, differ at bit 1 (expected `0`, actual `1`), and the last bit is then coded with the plain tree at index `10101101` = 173. (The first six rows of the table are the same as above, the seventh has index $256 + 86 = 342$ with the actual bit `1`.)

6.3 Why this helps

After a match ends, the next byte is not random: it is the byte that *almost* continued the match. Differences between similar strings are often in the low bits (for instance `X` vs. `Y`, or a changed digit). Coding the high bits "relative to" the match byte spends far fewer bits than coding the byte from scratch. This is one of the main single sources of LZMA's advantage over Deflate, which has no such mechanism.

Chapter 7

Match Lengths and Distances

7.1 The length coder

Lengths range from 2 to 273 ($= 18 + 255$). They are split into three ranges, each with its own model; a *choice* bit and a *choice2* bit select the range. Short lengths are the most common and get a cheap, `posState`-dependent coding.

decisions	length range	model
`choice=0`	$2 \ldots 9$	3-bit tree `low[posState]`, value $L - 2$
`choice=1`, `choice2=0`	$10 \ldots 17$	3-bit tree `mid[posState]`, value $L - 10$
`choice=1`, `choice2=1`	$18 \ldots 273$	8-bit tree `high`, value $L - 18$

Worked example: coding lengths

- $L = 4$: `choice=0`; `low` tree value $4 - 2 = 2 =$ `010`. Cost: 4 decisions.
- $L = 12$: `choice=1`, `choice2=0`; `mid` tree value $12 - 10 = 2 =$ `010`. Cost: 5 decisions.
- $L = 100$: `choice=1`, `choice2=1`; `high` tree value $100 - 18 = 82 =$ `01010010`. Cost: 10 decisions.
- $L = 273$: `high` value $255 =$ `11111111` (the maximum).

There are two separate length coders: one for ordinary matches and one for repeated matches (they have different statistics).

7.2 The distance coder

The distance $D \geq 1$ is stored as $d = D - 1 \geq 0$. The key observation: distances are spread roughly logarithmically, and the exact low bits of a large distance are mostly noise, while the magnitude (the *number of bits*) is predictable. So LZMA splits d into a *slot* (describing the magnitude and the second-highest bit) and *extra bits*.

Slot. For $d < 4$, slot $= d$. For $d \geq 4$, let $n = \lfloor \log_2 d \rfloor$ (position of the highest set bit) and let b be the next bit below it:

$$\text{slot} = 2n + b.$$

The slot is a 6-bit value (0–63) coded with a 6-level bit tree, using one of 4 trees selected by `lenState` $= \min(L-2, 3)$. (Short matches have different distance statistics than long ones: a 2-byte match is only worthwhile at small distances.)

Extra bits. A slot ≥ 4 carries $n_x = (\text{slot} \gg 1) - 1$ extra bits, and the slot determines the base value $\text{base} = (2 \mid (\text{slot} \,\&\, 1)) \ll n_x$. The extra bits are $d - \text{base}$ (an n_x-bit number). How they are coded depends on the slot:

- **Slots 4–13** (distances < 128): all n_x bits with a *reverse bit tree* from `posSpecial` (adaptive).
- **Slots 14–63**: the top $n_x - 4$ bits as *direct bits* (probability 1/2, 1 bit each) and the low 4 bits with a reverse bit tree `align` (adaptive).

slot	n_x extra bits	range of $d = D - 1$	coding of extra bits
0–3	0	$d = \text{slot}$	none
4	1	4–5	reverse tree
5	1	6–7	reverse tree
6	2	8–11	reverse tree
7	2	12–15	reverse tree
8	3	16–23	reverse tree
10	4	32–47	reverse tree
12	5	64–95	reverse tree
13	5	96–127	reverse tree
14	6	128–191	2 direct + 4 align
19	8	768–1023	4 direct + 4 align
31	14	49152–65535	10 direct + 4 align
39	18	786432–1048575	14 direct + 4 align
63	30	$3 \cdot 2^{30}$–$2^{32} - 1$	26 direct + 4 align

Worked example: distance 5 and distance 1000

$D = 5$**:** $d = 4 =$ `100`, $n = 2$, next bit $b = 0$, slot $= 4$. $n_x = (4 \gg 1) - 1 = 1$, base $= 2 \ll 1 = 4$, extra $= 0$. Coded: slot `000100` in the slot tree for the given `lenState`, then the 1-bit reverse tree with value `0`.
$D = 1000$**:** $d = 999 =$ `1111100111` (10 bits). $n = 9$, next bit $b = 1$, slot $= 2 \cdot 9 + 1 = 19$. $n_x = (19 \gg 1) - 1 = 8$, base $= (2 \mid 1) \ll 8 = 768$, extra $= 999 - 768 = 231 =$ `11100111`. Since slot ≥ 14: the top $8 - 4 = 4$ extra bits `1110` are direct bits (4 bits cost) and the low 4 bits `0111` are coded LSB-first (`1,1,1,0`) in the `align` reverse tree. Total: 6 slot-tree decisions + 4 direct bits + 4 align decisions.

Why this design is smart

The slot tree learns the *distribution of distance magnitudes* (cheaply, since only 64 symbols). The align bits capture the common pattern that distances are often multiples of 2, 4, 8 or 16 in structured data. The direct bits are the truly unpredictable part and are sent essentially raw, at cost of exactly 1 bit each, without polluting the models.

7.3 End marker

Distance d = `0xFFFFFFFF` (slot 63, all extra bits 1) is not a valid distance; it is used as an *end-of-payload marker*. Streams of unknown length end with it (a match packet with this distance). Streams that store their size in the header may omit it.

Chapter 8

Repeated Matches

The decoder keeps the four most recently used distances `rep0`, `rep1`, `rep2`, `rep3` (initially all 0, meaning distance 1). Data with structure repeats *distances*: a table with fixed-size records, UTF-16 text, a source file where similar lines are always the same number of bytes back, or a match interrupted by a single changed byte and then resumed at the same distance.

- **MATCH** (new distance): shift the history, `rep3` ← `rep2` ← `rep1` ← `rep0` ← d.
- **LONGREP**[k]: use $\texttt{rep}_k$, move it to the front (the others keep relative order), and code only the *length*, with the separate repeated-match length coder.
- **SHORTREP**: copy exactly one byte from `rep0` (no length coded). This is the cheapest nontrivial packet, useful when the byte after a match happens to equal the byte at distance `rep0`, that is, a repeat match of length 1, which a LONGREP (minimum length 2) cannot express.

Worked example: rep distance cost

Coding a repeat of the previous match's distance takes, at 50/50 probabilities, 4 decisions for the packet type (`isMatch`=1, `isRep`=1, `isRepG0`=0, `isRep0Long`=1) plus the length (4 decisions for $L = 4$): 8 bits. A fresh match of $D = 1000$ would cost 2 (type) + 4 (length) + 6 (slot) + 8 (extra) = 20 bits. After adaptation the type decisions shrink to fractions of a bit, so a rep match can cost around 1–3 bits where a fresh match costs 15–25 bits.

This is why the encoder's optimal parser (Chapter 11) loves rep matches: a short rep match can be *cheaper than a single literal*.

Chapter 9

A Complete Worked Example

Let us now follow a real compression end to end. The input is the 15-byte string

`abcdXabcdYabcdZ`

compressed with the real liblzma (preset 6, `.lzma` format, defaults `lc` = 3, `lp` = 0, `pb` = 2). I decoded the output with my instrumented reference decoder to recover the exact packets the encoder chose.

9.1 The packets chosen by the encoder

pos	posState	state before	packet	bytes produced	state after
0	0	0	LIT `a`	`a`	0
1	1	0	LIT `b`	`b`	0
2	2	0	LIT `c`	`c`	0
3	3	0	LIT `d`	`d`	0
4	0	0	LIT `X`	`X`	0
5	1	0	MATCH $L = 4$, $D = 5$	`abcd`	7
9	1	7	LIT `Y` (matched, expected `X`)	`Y`	4
10	2	4	LONGREP[0], $L = 4$ ($D = 5$ again)	`abcd`	8
14	2	8	LIT `Z` (matched, expected `Y`)	`Z`	5

9.2 What was coded for each packet

1. **Pos 0–4, five literals.** For each: `isMatch[0][posState]`=0, then 8 bits through the literal tree of `litState` = prevByte $\gg$ 5. For `a` the previous byte is taken to be 0, so `litState` = 0; for the rest, the previous byte is `a`,`b`,`c`,`d`, with `0x61` $\gg$ 5 = `0x62` $\gg$ 5 = `0x63` $\gg$ 5 = `0x64` $\gg$ 5 = 3, so they all share context 3. (Letters share a context, which is exactly the point of using only high bits.)

2. **Pos 5, the match.** `isMatch[0][1]`=1; `isRep[0]`=0 (a new distance); length: `choice`=0 and `low[1]` value $4 - 2 = 2 =$ `010`; distance: $D = 5 \Rightarrow d = 4$, slot 4, with `lenState` = $\min(4 - 2, 3) = 2$, slot tree `posSlot[2]` coding `000100`, then the 1-bit reverse tree from `posSpecial` with value 0. State $0 \to 7$, `rep0` = 4.

3. **Pos 9, a literal in state 7.** `isMatch[7][1]`=0. State ≥ 7 so it is a *matched literal*: `matchByte` = out[9 − (`rep0` + 1)] = out[4] = `X`. This is exactly the table of Chapter 6. State

$7 \to 4$ (rule: $4 \le s < 10 \Rightarrow s - 3$).

4. **Pos 10, the rep match.** `isMatch[4][2]`=1, `isRep[4]`=1, `isRepG0[4]`=0, `isRep0Long[4][2]`=1; length via the *repeat-length* coder: `choice`=0, `low[2]`=`010`. State $4 \to 8$. (A fresh match with $D = 5$ would also have worked but is more expensive: it needs the slot and extra bits as well as the type bits.)

5. **Pos 14, last literal.** `isMatch[8][2]`=0; matched literal with `matchByte` = out[14 − 5] = `Y`; state $8 \to 5$.

6. **End marker.** Because this `.lzma` stream has unknown size in its header, the encoder appends a match packet with distance `0xFFFFFFFF`.

9.3 The actual bytes

The compressed file is 33 bytes:

```
5d 00 00 80 00 ff ff ff ff ff ff ff ff 00 30 98 88 98 3d fe f2 3b 80 dd d4 44 e0 7f ff
                                                                f8 1b 20 00
```

- `5d`: properties byte (`lc` $= 3$, `lp` $= 0$, `pb` $= 2$).
- `00 00 80 00`: dictionary size, little-endian = `0x00800000` = 8 MiB.
- eight `ff`: uncompressed size unknown ($2^{64} - 1$), so an end marker follows the data.
- the remaining 20 bytes are the range-coded payload. The first is the mandatory `0x00`.

Why did 15 bytes become 33?

Tiny inputs cannot be compressed: the 13-byte header alone nearly doubles it, and every probability starts at 1/2, so each early decision costs a full bit. A rough count: five literals $\approx 5 \times 9 = 45$ bits, the match ≈ 13 bits, the first matched literal ≈ 9, the rep ≈ 8, the last literal ≈ 9, and the end marker ≈ 42 (type, length, slot and 30 distance bits). That is about 126 bits ≈ 16 bytes, and the flush of the range coder adds the rest of the 20 bytes. Adaptive modelling only pays off once the models have been trained on a few kilobytes. The gains in Chapter 12 are measured on megabytes.

You can now follow the example in Appendix A: save these 33 bytes as a file and run the C decoder on it; it writes out `abcdXabcdYabcdZ`. A good exercise is to add a `printf` to the decoder's main loop and watch it print exactly the packet table above.

Chapter 10

Container Formats

The range-coded payload described so far is wrapped in one of several containers.

10.1 The `.lzma` (LZMA-alone) format

A 13-byte header followed by the range-coded data:

bytes	field	notes
1	properties	$(\texttt{pb} \cdot 5 + \texttt{lp}) \cdot 9 + \texttt{lc}$, typically `0x5D`
4	dictionary size	little-endian; typically a power of two
8	uncompressed size	little-endian; all `0xFF` means unknown (end marker required)

It has no checksum and no framing, so it is mostly of historical interest; but it is exactly the format that Appendix A decodes.

10.2 LZMA2

LZMA2 is a thin framing layer around LZMA that fixes three weaknesses: (i) incompressible data is expanded slightly by LZMA, (ii) the stream cannot be split for parallelism, (iii) there is no way to change parameters mid-stream. An LZMA2 stream is a series of *chunks*, each introduced by a control byte:

control byte	meaning
`0x00`	end of stream
`0x01`	uncompressed chunk, with dictionary reset
`0x02`	uncompressed chunk, no reset
`0x80`–`0xFF`	LZMA chunk. Bits 6–5 select what is reset: none / state / state + new properties / state + new properties + dictionary. Bits 4–0 are the high bits of the chunk's uncompressed size minus one.

An LZMA chunk then carries the rest of its uncompressed size, its compressed size, and optionally a new properties byte, followed by range-coded data. The range coder is re-initialised for every LZMA chunk (a chunk covers at most about 2 MiB of uncompressed data and 64 KiB of compressed data). Uncompressed chunks hold at most 64 KiB. The encoder can fall back to an

uncompressed chunk whenever LZMA would expand the data, and independent *dictionary-reset* points allow multi-threaded compression of blocks.

10.3 `.xz` and `.7z`

The `.xz` format wraps LZMA2 in a container with a magic number, optional integrity check (CRC32, CRC64 or SHA-256), block structure, an index for random access to blocks, and a chain of *filters*. Filters such as BCJ (branch/call/jump) convert relative addresses in executable code to absolute ones before LZMA2, making repeated calls to the same function byte-identical and thus matchable. The `.7z` format is a different container (supporting solid archives and per-folder filter chains) that uses LZMA or LZMA2 inside.

Chapter 11

The Encoder Side

The format fixes the decoder precisely; the encoder is free. Two components dominate an LZMA encoder: the *match finder* and the *optimal parser*.

11.1 Match finders

For every position the encoder needs candidate matches in the dictionary, namely for each achievable length the *closest* distance (closer distances are cheaper to code).

Hash chains (`hc3`, `hc4`). A hash of the next 3 or 4 bytes indexes a table that stores the most recent position with that hash; an array `prev[pos]` links each position to the previous one with the same hash. To find matches, walk the chain from newest to oldest for at most `depth` steps and compare the bytes. Simple and fast, but with limited depth it can miss long-range matches.

Binary trees (`bt2`, `bt3`, `bt4`). All positions sharing a hash are kept in a binary search tree ordered by the string that follows each position. A single descent from the root, guided by comparisons, discovers matches of strictly increasing length (and, because newer positions are inserted nearer to the root, decreasing distance), and inserts the current position at the same time. It is slower per position than a chain walk but finds essentially all useful matches. The cost is memory: roughly eleven to twelve times the dictionary size for `bt4` in the `xz` implementation.

11.2 Optimal parsing

Imagine every position $0, 1, 2, \ldots$ in a window as a node, and every available packet as an edge from the position where it starts to the position where it ends, weighted by its *cost in bits* (computed from the current adaptive probabilities as $-\log_2 p$ per decision; the real implementation uses small lookup tables with fractional-bit precision). The cheapest way to encode the window is the shortest path from the first node to the last, which is a dynamic-programming problem:

$$\text{cost}[j] = \min_{\text{edges } i \to j} \left(\text{cost}[i] + \text{price}(\text{edge})\right).$$

A practical optimal parser (the one in the LZMA SDK) works as follows:

1. At the current position, gather candidates: literal; all rep matches (rep0..rep3) with their lengths; all matches from the match finder (for each length, the nearest distance); and compound patterns such as “literal followed by rep0 match” or “match, literal, rep0 match”.

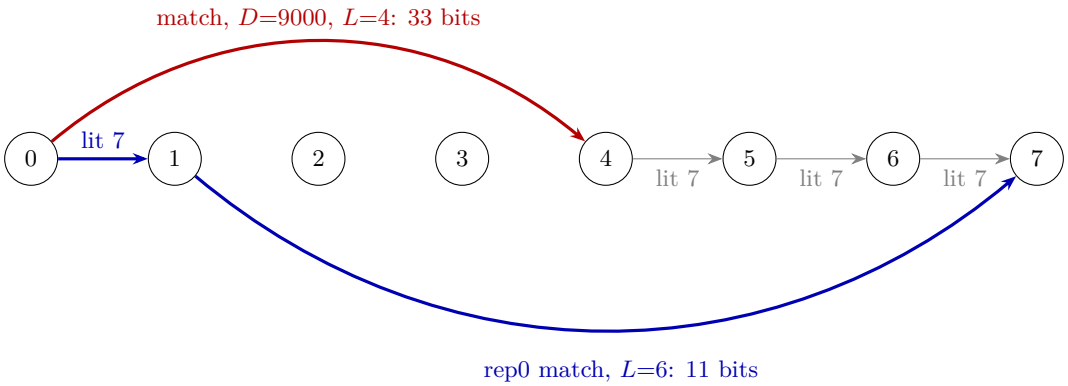


Figure 11.1: A greedy parser takes the longest match at position 0 (red: 33 bits to reach position 4) and then must finish with literals: $33 + 7 + 7 + 7 = 54$ bits. The optimal path (blue) takes one literal and then a repeated match: $7 + 11 = 18$ bits. The costs are illustrative, but costs of this magnitude (a fresh match with a far distance is expensive, a rep match is cheap) are realistic.

2. Extend the search forward through the cheapest reachable positions, recording for each the best known cost and the packet and state that achieved it, until the window ends or a very long match (`nice_len`) is found.

3. Walk the best path back from the end and *emit* those packets, updating the real probabilities and state, then continue from where the path ended.

It is a heuristic, not a perfect shortest path: the price of an edge depends on the state and the rep history, which depend on the path taken, and the adaptive probabilities change as packets are coded. Nevertheless it yields substantial gains over greedy parsing, as measured below.

11.3 Presets

`xz` presets 0–3 use a faster mode with hash-chain match finders and simpler parsing; presets 4–9 use binary trees and optimal parsing. Dictionary sizes and approximate memory (from the `xz` manual; they may vary slightly between versions):

preset	dictionary	compressor memory	decompressor memory
0	256 KiB	3 MiB	1 MiB
1	1 MiB	9 MiB	2 MiB
3	4 MiB	32 MiB	5 MiB
6	8 MiB	94 MiB	9 MiB
9	64 MiB	674 MiB	65 MiB

Note the asymmetry: decompression needs memory roughly equal to the dictionary (the sliding window) plus about 16 KiB of probabilities, while compression needs about an order of magnitude more.

Chapter 12

Why LZMA Wins (and Where It Loses)

12.1 Measured comparison

All numbers below were measured on one machine with Python's standard `zlib`, `bz2` and `lzma` modules, on two data sets. Timings are indicative only; the point is the ratio of sizes.

Data set A (text). 8 000 000 bytes: 574 Python source files concatenated.

compressor	size (bytes)	ratio	compression time
zlib level 6 (Deflate)	1 770 042	22.13%	0.26 s
zlib level 9 (Deflate)	1 746 846	21.84%	1.02 s
bzip2 level 9	1 373 963	17.17%	0.62 s
LZMA preset 1	1 538 776	19.23%	0.49 s
LZMA preset 6	**1 253 160**	**15.66%**	4.76 s

Decompression of this set: Deflate 0.03 s, LZMA 0.11 s.

Data set B (binaries). 6 000 000 bytes from executables in `/usr/bin`.

compressor	size (bytes)	ratio	compression time
zlib level 9	2 838 509	47.31%	0.78 s
bzip2 level 9	2 675 845	44.60%	0.51 s
LZMA preset 6	**2 312 984**	**38.55%**	3.49 s

On the text LZMA produces output 28% smaller than Deflate-9 and 9% smaller than bzip2; on the binaries, 19% and 14% smaller respectively.

12.2 Which ingredient matters how much?

Using the real encoder with settings changed one at a time (data set A, LZMA2 raw streams, otherwise preset 6):

configuration	size (bytes)	ratio
default (dictionary 8 MiB, `lc` = 3, `lp` = 0, `pb` = 2, optimal parsing)	1 253 098	15.66%
dictionary 32 KiB (the Deflate window)	1 575 174	19.69%
dictionary 256 KiB	1 365 127	17.06%
dictionary 1 MiB	1 302 138	16.28%
fast mode, `hc4`, `nice_len`=64 (no optimal parsing)	1 476 349	18.45%
normal mode, `nice_len`=8 (short search)	1 537 379	19.22%
normal mode, `nice_len`=273	1 249 222	15.62%
`lc` = 0	1 257 114	15.71%
`lc` = 4	1 252 291	15.65%
`pb` = 0	1 250 636	15.63%

Observations (all specific to this data set, and worth repeating on your own data):

- **The dictionary matters a lot** on this data: shrinking it to Deflate's 32 KiB costs 26% more output.
- **Search and parsing effort matter a lot**: the fast mode is 18% larger than the optimal-parsing mode, and a shallow search is 23% larger.
- **`lc`, `lp`, `pb` matter little for source text** (differences of a few tenths of a percent) but can matter much more for structured binary data. Context bits are not magic; each costs learning time.

The remaining advantage over Deflate comes from the range coder and the context modelling (flags conditioned on state and position, matched literals, rep matches), which does not show up in an ablation of tuning parameters but is built into the format. It is why even a 32 KiB-dictionary LZMA stream is smaller than a Deflate stream with a similar window (1 575 174 bytes against 1 746 846 here).

12.3 The summary

	Deflate (gzip)	LZMA
Window	32 KiB	MiB to GiB
Entropy coder	Huffman trees per block, sent in the stream	adaptive binary range coder, no side information
Cost floor per decision	1 bit	about 0.02 bits
Literal modelling	none (a single tree)	context: previous byte, position bits, expected byte after a match
Distance modelling	fixed code classes, extra bits raw	slot tree by length class, align bits, 4 repeat distances
Parsing	greedy / lazy	optimal (price-based dynamic programming)
Decoder speed	very fast	moderate (serial bit-by-bit)
Compression speed	fast	slow

12.4 Limitations

- **Decoding is inherently serial.** Each decision depends on the adaptive state left by all previous ones. You cannot start decoding in the middle of a chunk, which is why LZMA2

and `xz` introduce independent blocks for multi-threading.

- **Slower than modern LZ codecs.** Zstandard and LZ4 decode several times faster, with Zstandard at its higher levels coming close to LZMA ratios on many data sets. LZMA's niche is "compress once, ship many times, minimise size".
- **Compressor memory** can be hundreds of MiB at high presets.
- **Incompressible data** is expanded slightly by plain LZMA (LZMA2 avoids this with stored chunks).
- **Not suitable for very small inputs**, since the models start untrained (Chapter 9).

Chapter 13

Exercises

Solutions follow. Try them first; use the decoder in Appendix A for experiments.

E1. Range coder step. The encoder has `range` = `0x10000000`, `prob` = 1500, and codes a `1`. Compute `bound`, the new `range`, the new `prob`, and the increase of `low`. Is normalisation needed?

E2. Distance coding. Code the distance $D = 300$: give d, the slot, the number of extra bits, the base, the extra value, and how the extra bits are coded.

E3. State machine. Starting at state 0, the packets are LIT, MATCH, LIT, LIT, LONGREP, SHORTREP. Give the state after each packet.

E4. Properties byte. (a) Decode the properties byte `0x5D`. (b) Compute the byte for `lc` = 0, `lp` = 2, `pb` = 2.

E5. Memory of the model. How many probabilities does a decoder with `lc` = 3, `lp` = 0 need? How many bytes if each is 16 bits?

E6. Length coding. Describe the decisions coded for $L = 2$, $L = 9$, $L = 10$, $L = 18$ and $L = 273$.

E7. Cost of a decision. A flag has probability 1990/2048 of being `0`. What does coding a `0` cost, and a `1`?

E8. Parsing. The text `xyzxyzxyzxyz` (12 bytes) is given. Find a short packet sequence. How does the overlapping-copy rule help?

E9. Thinking question. Why does the range coder use only the *high* bits of `range` to compute `bound`, and why must the probability never reach 0 or 2048?

E10. Challenge. Modify the C decoder in Appendix A to count how many packets of each type (LIT, MATCH, SHORTREP, LONGREP[0–3]) a stream contains. Run it on files produced by `xz -format=lzma` for some text file and for an executable. What is the typical ratio of literals to matches? How common are rep matches?

Solutions

S1. `range` $\gg 11 = 2^{28}/2^{11} = 131\,072$; `bound` $= 131\,072 \times 1500 = 196\,608\,000$. Bit `1`: `low` increases by $196\,608\,000$; `range` $= 268\,435\,456 - 196\,608\,000 = 71\,827\,456$; `prob` $= 1500 - (1500 \gg 5) = 1500 - 46 = 1454$. Since $71\,827\,456 > 2^{24} = 16\,777\,216$, no normalisation.

S2. $d = 299 =$ `100101011` (9 bits), $n = 8$, next bit 0, slot $= 16$. $n_x = (16 \gg 1) - 1 = 7$, base $= 2 \ll 7 = 256$, extra $= 299 - 256 = 43 =$ `0101011`. Slot ≥ 14: top $7 - 4 = 3$ bits `010` as direct bits, low 4 bits `1011` with the `align` reverse tree (coded LSB first: 1,1,0,1).

S3. LIT: 0; MATCH: 7; LIT: $7 \to 4$; LIT: $4 \to 1$; LONGREP: $1 < 7 \to 8$; SHORTREP: $8 \geq 7 \to 11$. Sequence: 0, 7, 4, 1, 8, 11.

S4. (a) $93 = (\texttt{pb} \cdot 5 + \texttt{lp}) \cdot 9 + \texttt{lc}$: $93 = 10 \cdot 9 + 3$, so `lc` $= 3$, and $10 = \texttt{pb} \cdot 5 + \texttt{lp}$ gives `pb` $= 2$, `lp` $= 0$. (b) $(2 \cdot 5 + 2) \cdot 9 + 0 = 108 =$ `0x6C`.

S5. $1846 + 768 \cdot 2^{\texttt{lc}+\texttt{lp}} = 1846 + 768 \cdot 8 = 7990$ probabilities; $\times 2$ bytes $= 15\,980$ bytes, about 15.6 KiB. (The 1846 base: $192 + 48 + 192 + 256 + 114 + 16 + 1028$ for the `isMatch`, rep flags, `isRep0Long`, slot trees, special positions, align, and two length coders; see Appendix B.)

S6. $L = 2$: `choice`=0, `low` value 0 (`000`). $L = 9$: `choice`=0, `low` value 7 (`111`). $L = 10$: `choice`=1, `choice2`=0, `mid` value 0. $L = 18$: `choice`=1, `choice2`=1, `high` value 0 (8 zeros). $L = 273$: `choice`=1, `choice2`=1, `high` value 255.

S7. $P(0) = 1990/2048 = 0.9717$: coding `0` costs $-\log_2 0.9717 = 0.041$ bits; coding `1` has probability $58/2048 = 0.0283$ and costs $-\log_2 0.0283 = 5.14$ bits. A confident model is cheap when right, and expensive (but not catastrophic) when wrong.

S8. Literals `x y z`, then one match with $D = 3$ and $L = 9$: $3 + 1$ packets for 12 bytes. Because the copy goes byte by byte, the match source overlaps the destination ($L > D$), and the 3-byte pattern repeats itself.

S9. The product (`range` $\gg 11$) $\times$ `prob` fits in 32 bits because `prob` $< 2^{11}$. If `prob` could reach 0 or 2048, `bound` would be 0 or `range`, making one outcome impossible (a zero-width interval) and the stream undecodable if that outcome occurred. The update rule's integer rounding keeps `prob` $\in [31, 2017]$.

S10. (Open-ended.) For ordinary text expect matches to be a minority of packets but to cover most of the bytes; rep matches are typically a significant fraction of matches in source code and structured data.

Appendix A

A Complete LZMA Decoder in C

The following decoder handles the `.lzma` (“LZMA-alone”) format and is about 200 lines of plain C99 with no dependencies. I tested it against files produced by the real `xz` tool (which uses liblzma) for presets `-0`, `-3`, `-6` and `-9e` on seven inputs (the 15-byte sample string, an empty file, 20 000 random bytes, a 100 000-byte run, repeated source text, 600 KB of executable code, and a 4.6 MB collection of source files), and with non-default settings (`lc`/`lp`/`pb` = 0/2/2, 4/0/0 and 1/3/4). Every output matched the original byte for byte, and all runs were clean under AddressSanitizer and UBSan. It favours clarity over speed (the whole output is kept in memory and serves as the dictionary), and every part corresponds to a section above.

code	explained in
`rc_bit`, `normalize`	Chapter 4 (decoding one bit, adaptation, normalisation)
`rc_direct`, `rc_tree`, `rc_rtree`	Chapter 4 (direct bits, bit trees, reverse trees)
`LenDec`, `len_decode`	Chapter 7 (length coder)
main loop, `state` updates	Chapter 5 (packets, state machine)
matched literal branch	Chapter 6
`slot`, `dist` computation	Chapter 7 (distance coder)
`rep[]` handling	Chapter 8

Listing A.1: `lzma_dec.c`: reference LZMA decoder (tested).

```c
/* lzma_dec.c - minimal reference decoder for the .lzma ("LZMA-alone") format.
 * Clarity over speed. Build:  gcc -O2 -Wall -o lzma_dec lzma_dec.c
 * Usage: ./lzma_dec input.lzma output.bin                                  */
#include <stdint.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef uint16_t Prob;
#define TOP_VALUE (1u << 24)

static void die(const char *msg) { fprintf(stderr, "error: %s\n", msg); exit(1); }

static void init_probs(Prob *p, size_t n) { while (n--) *p++ = 1024; }

/* ---------------- range decoder (Chapter 4) ---------------- */
typedef struct { const uint8_t *in, *end; uint32_t range, code; } RC;

static uint8_t next_byte(RC *rc) {
    if (rc->in >= rc->end) die("truncated input");
```

```
    return *rc->in++;
}
static void rc_init(RC *rc, const uint8_t *in, const uint8_t *end) {
    rc->in = in; rc->end = end; rc->range = 0xFFFFFFFFu; rc->code = 0;
    if (next_byte(rc) != 0) die("first range-coder byte must be 0");
    for (int i = 0; i < 4; i++) rc->code = (rc->code << 8) | next_byte(rc);
}
static void normalize(RC *rc) {
    if (rc->range < TOP_VALUE) {
        rc->range <<= 8;
        rc->code = (rc->code << 8) | next_byte(rc);
    }
}
/* decode one bit with adaptive probability *p */
static int rc_bit(RC *rc, Prob *p) {
    uint32_t bound = (rc->range >> 11) * *p;
    int bit;
    if (rc->code < bound) { rc->range = bound;  *p += (2048 - *p) >> 5; bit = 0; }
    else { rc->range -= bound; rc->code -= bound; *p -= *p >> 5;        bit = 1; }
    normalize(rc);
    return bit;
}
/* n bits with fixed probability 1/2 */
static uint32_t rc_direct(RC *rc, int n) {
    uint32_t r = 0;
    while (n--) {
        rc->range >>= 1;
        uint32_t b = rc->code >= rc->range;
        if (b) rc->code -= rc->range;
        r = (r << 1) | b;
        normalize(rc);
    }
    return r;
}
/* bit tree, MSB first; probs has 2^nbits entries (index 0 unused) */
static uint32_t rc_tree(RC *rc, Prob *probs, int nbits) {
    uint32_t m = 1;
    for (int i = 0; i < nbits; i++) m = (m << 1) | rc_bit(rc, &probs[m]);
    return m - (1u << nbits);
}
/* reverse bit tree, LSB first */
static uint32_t rc_rtree(RC *rc, Prob *probs, int nbits) {
    uint32_t m = 1, sym = 0;
    for (int i = 0; i < nbits; i++) {
        uint32_t b = rc_bit(rc, &probs[m]);
        m = (m << 1) | b;
        sym |= b << i;
    }
    return sym;
}

/* ---------------- length coder (Chapter 7) ---------------- */
typedef struct { Prob choice, choice2, low[16][8], mid[16][8], high[256]; } LenDec;

static void len_init(LenDec *l) { init_probs((Prob *)l, sizeof *l / sizeof(Prob)); }
static uint32_t len_decode(RC *rc, LenDec *l, unsigned pos_state) {
    if (!rc_bit(rc, &l->choice))  return 2  + rc_tree(rc, l->low[pos_state], 3);
    if (!rc_bit(rc, &l->choice2)) return 10 + rc_tree(rc, l->mid[pos_state], 3);
    return 18 + rc_tree(rc, l->high, 8);
}

/* ---------------- output buffer (the whole output is the dictionary) ---------------- */
typedef struct { uint8_t *buf; size_t len, cap; } Out;
```

```c
static void out_put(Out *o, uint8_t b) {
    if (o->len == o->cap) {
        o->cap = o->cap ? o->cap * 2 : 4096;
        o->buf = realloc(o->buf, o->cap);
        if (!o->buf) die("out of memory");
    }
    o->buf[o->len++] = b;
}

/* ---------------- main decoder ---------------- */
static Out lzma_alone_decode(const uint8_t *data, size_t size) {
    if (size < 13) die("file too short");
    unsigned d = data[0];
    if (d >= 9 * 5 * 5) die("bad properties byte");
    unsigned lc = d % 9; d /= 9;
    unsigned lp = d % 5; unsigned pb = d / 5;
    uint64_t usize = 0;
    for (int i = 0; i < 8; i++) usize |= (uint64_t)data[5 + i] << (8 * i);
    int unknown = (usize == UINT64_MAX);

    RC rc; rc_init(&rc, data + 13, data + size);

    size_t nlit = (size_t)0x300 << (lc + lp);
    Prob *literal = malloc(nlit * sizeof(Prob));
    if (!literal) die("out of memory");
    init_probs(literal, nlit);

    Prob is_match[12][16], is_rep[12], is_rep_g0[12], is_rep_g1[12], is_rep_g2[12];
    Prob is_rep0_long[12][16], pos_slot[4][64], pos_special[115], align[16];
    init_probs(&is_match[0][0], 12 * 16);  init_probs(is_rep, 12);
    init_probs(is_rep_g0, 12); init_probs(is_rep_g1, 12); init_probs(is_rep_g2, 12);
    init_probs(&is_rep0_long[0][0], 12 * 16);
    init_probs(&pos_slot[0][0], 4 * 64);   init_probs(pos_special, 115);
    init_probs(align, 16);
    LenDec len_dec, rep_len_dec; len_init(&len_dec); len_init(&rep_len_dec);

    Out out = {0};
    unsigned state = 0;
    uint32_t rep[4] = {0, 0, 0, 0};

    while (unknown || out.len < usize) {
        size_t pos = out.len;
        unsigned ps = pos & ((1u << pb) - 1);

        if (!rc_bit(&rc, &is_match[state][ps])) {                 /* ---- literal */
            unsigned prev = pos ? out.buf[pos - 1] : 0;
            Prob *probs = literal + 0x300 * (((pos & ((1u << lp) - 1)) << lc) + (prev >> (8 -
 lc)));
            unsigned sym = 1;
            if (state >= 7) {                                      /* matched literal */
                unsigned mb = out.buf[pos - rep[0] - 1];
                while (sym < 0x100) {
                    unsigned match_bit = (mb >> 7) & 1; mb <<= 1;
                    unsigned bit = rc_bit(&rc, &probs[((1 + match_bit) << 8) + sym]);
                    sym = (sym << 1) | bit;
                    if (match_bit != bit) break;
                }
            }
            while (sym < 0x100) sym = (sym << 1) | rc_bit(&rc, &probs[sym]);
            out_put(&out, (uint8_t)sym);
            state = state < 4 ? 0 : (state < 10 ? state - 3 : state - 6);
            continue;
        }
```

```
146
147         uint32_t len;
148         if (!rc_bit(&rc, &is_rep[state])) {                         /* ---- simple match */
149             rep[3] = rep[2]; rep[2] = rep[1]; rep[1] = rep[0];
150             len = len_decode(&rc, &len_dec, ps);
151             state = state < 7 ? 7 : 10;
152             uint32_t slot = rc_tree(&rc, pos_slot[len - 2 < 3 ? len - 2 : 3], 6);
153             uint32_t dist;
154             if (slot < 4) dist = slot;
155             else {
156                 int nd = (slot >> 1) - 1;
157                 dist = (2 | (slot & 1)) << nd;
158                 if (slot < 14) dist += rc_rtree(&rc, pos_special + dist - slot, nd);
159                 else {
160                     dist += rc_direct(&rc, nd - 4) << 4;
161                     dist += rc_rtree(&rc, align, 4);
162                 }
163             }
164             if (dist == 0xFFFFFFFFu) break;                         /* end marker */
165             rep[0] = dist;
166         } else {                                                    /* ---- rep match */
167             if (!rc_bit(&rc, &is_rep_g0[state])) {
168                 if (!rc_bit(&rc, &is_rep0_long[state][ps])) {       /* short rep */
169                     state = state < 7 ? 9 : 11;
170                     if (rep[0] >= pos) die("bad distance");
171                     out_put(&out, out.buf[pos - rep[0] - 1]);
172                     continue;
173                 }
174             } else {
175                 uint32_t dist;
176                 if (!rc_bit(&rc, &is_rep_g1[state])) dist = rep[1];
177                 else {
178                     if (!rc_bit(&rc, &is_rep_g2[state])) dist = rep[2];
179                     else { dist = rep[3]; rep[3] = rep[2]; }
180                     rep[2] = rep[1];
181                 }
182                 rep[1] = rep[0]; rep[0] = dist;
183             }
184             len = len_decode(&rc, &rep_len_dec, ps);
185             state = state < 7 ? 8 : 11;
186         }
187         if (rep[0] >= pos) die("bad distance");
188         if (!unknown && len > usize - out.len) die("match exceeds declared size");
189         for (uint32_t i = 0; i < len; i++)                          /* byte-by-byte copy */
190             out_put(&out, out.buf[out.len - rep[0] - 1]);
191     }
192     free(literal);
193     return out;
194 }
195
196 int main(int argc, char **argv) {
197     if (argc != 3) { fprintf(stderr, "usage: %s in.lzma out\n", argv[0]); return 2; }
198     FILE *f = fopen(argv[1], "rb");
199     if (!f) die("cannot open input");
200     fseek(f, 0, SEEK_END); long n = ftell(f); fseek(f, 0, SEEK_SET);
201     uint8_t *data = malloc(n > 0 ? n : 1);
202     if (!data || fread(data, 1, n, f) != (size_t)n) die("read failed");
203     fclose(f);
204     Out out = lzma_alone_decode(data, (size_t)n);
205     FILE *g = fopen(argv[2], "wb");
206     if (!g) die("cannot open output");
207     if (out.len && fwrite(out.buf, 1, out.len, g) != out.len) die("write failed");
208     fclose(g);
```

```
    free(out.buf); free(data);
    return 0;
}
```

Building and testing.

```
gcc -O2 -Wall -o lzma_dec lzma_dec.c
echo -n "abcdXabcdYabcdZ" > sample.txt
xz --format=lzma -c sample.txt > sample.lzma     # 33 bytes, as in Chapter 9
./lzma_dec sample.lzma roundtrip.txt
cmp sample.txt roundtrip.txt && echo identical
```

Appendix B

Cheat Sheet of Constants and Contexts

B.1 Probability arrays

All probabilities are 11-bit, initialised to 1024; adaptation shift is 5.

array	indexed by	number of probabilities
`isMatch`	state (12), posState (16)	192
`isRep`, `isRepG0`, `isRepG1`, `isRepG2`	state (12)	$4 \times 12 = 48$
`isRep0Long`	state (12), posState (16)	192
`posSlot`	lenState (4), 6-bit tree (64)	256
`posSpecial`	reverse trees for slots 4–13	114
`align`	4-bit reverse tree	16
length coder (two of them)	choice 2, low 16×8, mid 16×8, high 256	$2 \times 514 = 1028$
subtotal (independent of `lc`, `lp`)		1846
`literal`	$2^{\texttt{lc}+\texttt{lp}}$ contexts $\times$ `0x300`	$768 \cdot 2^{\texttt{lc}+\texttt{lp}}$

(The number of posState values actually used is $2^{\texttt{pb}}$; the arrays are sized for the maximum, $\texttt{pb} = 4$.)

B.2 Numeric constants

probability bits	11 (probabilities in $[0, 2048)$, start 1024)
adaptation shift	5
probability limits in practice	31 to 2017
range-coder top value	2^{24} (normalise when `range` falls below it)
initial range	`0xFFFFFFFF`
minimum / maximum match length	2 / 273
state count	12 (literal states 0–6, match states 7–11)
distance slots	64 (slots < 4: no extra bits; 4–13: modelled; ≥ 14: direct + 4 align)
end-of-payload marker	distance `0xFFFFFFFF` (slot 63, all extra bits 1)
default properties	$\texttt{lc} = 3$, $\texttt{lp} = 0$, $\texttt{pb} = 2$ = byte `0x5D`

Appendix C

Further Reading

- Igor Pavlov, *LZMA SDK* and the accompanying `lzma-specification` document (public domain); the C and C++ sources are the ultimate reference.
- Lasse Collin, *The .xz File Format* specification, and the `xz`/liblzma sources (the encoder, match finders and presets).
- J. Ziv and A. Lempel, "A Universal Algorithm for Sequential Data Compression", *IEEE Transactions on Information Theory*, 1977 (LZ77).
- C. E. Shannon, "A Mathematical Theory of Communication", 1948.
- G. N. N. Martin, "Range encoding: an algorithm for removing redundancy from a digitised message", 1979; and J. Rissanen and G. Langdon, "Arithmetic coding", *IBM Journal of Research and Development*, 1979.
- Matt Mahoney, *Data Compression Explained*, an online book covering context modelling and arithmetic coding.